



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SEMESTER 2

2025/2026

SECP3843-02 SPECIAL TOPIC IN DATA ENGINEERING

Tutorial 4 - Claude & Pipeline

Made by:

NAME	MATRIC NUMBER
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

LECTURER: DR. ARYATI BINTI BAKRI

1.0 AI-assisted Data Engineering

AI-assisted data engineering refers to the use of artificial intelligence agents to help design, build, and manage data pipelines with minimal manual coding or technical configuration. Instead of writing code from scratch or navigating complex user interfaces, users can interact with an AI agent through conversation to set up data sources, apply transformation logic, schedule updates, and debug issues automatically. As demonstrated in the video, tools like Nexla Express allow a complete ETL pipeline from pulling data via an API to loading it into a data warehouse like Snowflake to be built entirely through a chat-based AI interface which makes data engineering more accessible and efficient (Kretz A., 2025).

2.0 The AI Agent Used in This Tutorial

This section introduces the AI agent used to build the ETL pipeline in this tutorial, describes its role at each stage of the pipeline, and outlines the AI models that underpin its functionality.

2.1 Overview of the Nexla Express Agent

The AI agent used in this tutorial is the Express agent provided by Nexla, a data operations platform that enables users to construct end-to-end ETL pipelines through a conversational interface. Rather than manually configuring connectors and transformation scripts, a user interacts with the Express agent by typing instructions in everyday English, and the agent translates these instructions into a working pipeline. The platform is accessed through a web browser at dataops.nexla.io.

Within Nexla, the Express agent functions as an orchestrator. It interprets the intent behind a user's request, retrieves the configuration templates for the relevant connectors, manages the credentials required to access each system, applies the necessary data transformations, and sets up the destination where the processed data will be stored. All of these actions take place within a single guided conversation, which removes the need for the user to navigate multiple separate configuration screens.

2.2 Role of the AI Agent within the Pipeline

In the context of this tutorial, the Express agent performs six distinct functions. Each function corresponds to a stage in the construction of the ETL pipeline, and together they illustrate how an AI agent can manage an entire data integration task from start to finish.

- Pipeline planning: the agent interprets the natural language request and produces a step-by-step plan, including the identification of the two connectors required by both the OpenWeather API and Snowflake.
- Credential management: the agent guides the user through the entry of API keys and database login details, and validates each connection before continuing.
- Data source creation: the agent configures the OpenWeather API as a data source, set to retrieve weather data for Kuala Lumpur every two minutes.
- Transformation design: the agent examines the structure of the incoming data, identifies the relevant field, and proposes a transformation. In this case, converting a temperature value from Kelvin to Fahrenheit.
- Destination setup: the agent configures the Snowflake destination, including the target database, warehouse, schema, table, and the mapping of fields to that table.
- Verification: before the pipeline is finalised, the agent checks each component, the transformation and the destination to confirm that it functions correctly.

2.3 AI Models Underlying the Agent

The Express agent is described as model-agnostic, meaning that it can be powered by different large language models, including those developed by OpenAI, and Google. For the purposes of this tutorial, the specific underlying model is not material; what is demonstrated is how the agent, regardless of the model behind it, is able to translate natural language prompts into a functioning ETL pipeline.

3.0 Use Case Diagram

Figure 3.1 presents a Use Case diagram outlining the core interactions between the system user and the Nexla Express AI Agent system boundary. It represents an end-to-end operational workflow where the user is capable of submitting pipeline requests, managing secure system credentials and configuring both the data sources and target destinations. Additionally, the

diagram defines the system capabilities for designing custom data transformations, verifying pipeline setups and continuously monitoring both the active pipeline execution and the structured data inside the warehouse environment.

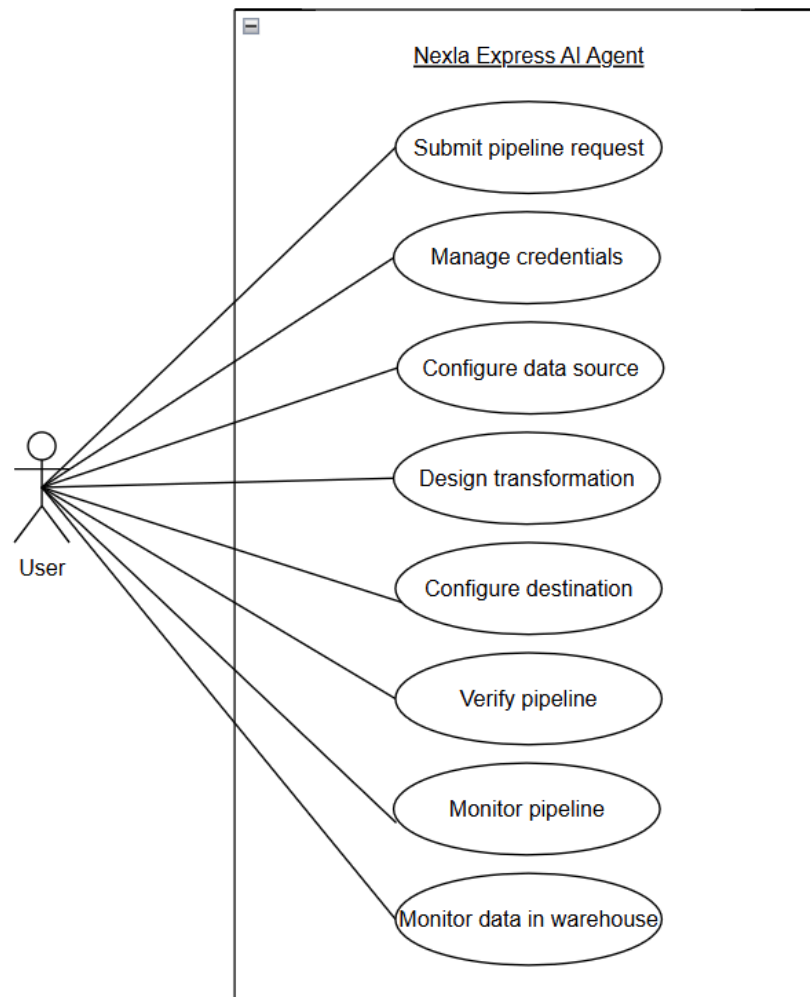


Figure 3.1: Use Case Diagram for Nexla Express AI Agent

4.0 Steps to Build the ETL Pipeline

This section describes the step-by-step process undertaken to build the ETL pipeline using the Nexla Express agent, from the initial natural language prompt to the verification of data in Snowflake. Each step is illustrated with the corresponding screenshot from the agent interface.

4.1 Step 1 - Initiating the Pipeline with a Natural Language Prompt

The pipeline was created by opening the Express agent interface and entering a single instruction describing the desired pipeline. The instruction submitted was: “I want to create a pipeline from the weather API at OpenWeather.com to my Snowflake table WEATHER_DATA. It should run every two minutes.”

Upon receiving this instruction, the agent began processing the request. It loaded the configuration templates for both systems named in the prompt – OpenWeather and Snowflake – confirming that it recognised both as valid connectors. Figure 4.1 shows this initial response from the agent.

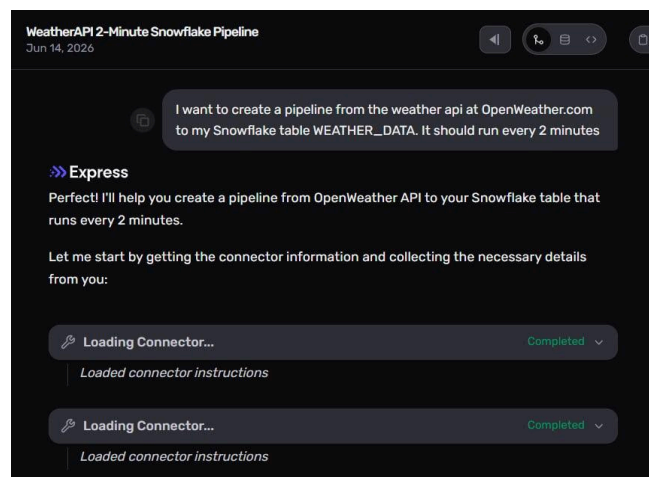


Figure 4.1: Express agent processing the pipeline request

4.2 Step 2- Credential Configuration

After the connectors were loaded, the agent checked whether credentials for OpenWeather and Snowflake already existed within the account. It detected an existing OpenWeather credential associated with London and an existing Snowflake credential named DEMO_DB. Figure 4.2 shows the options presented by the agent at this stage either to reuse the existing credentials or to create new ones.

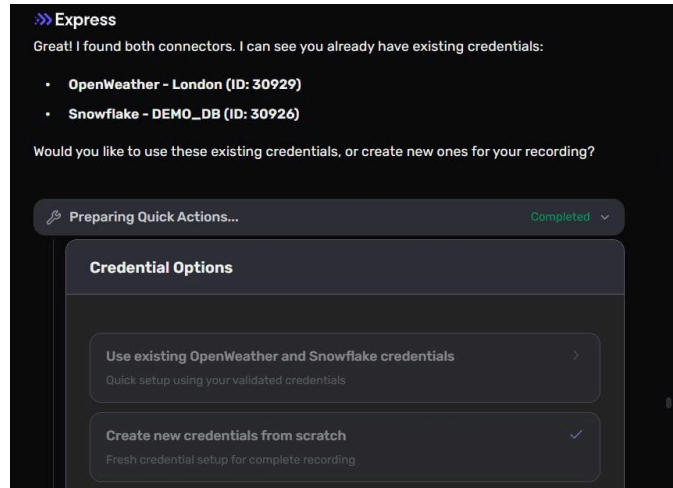


Figure 4.2: Agent presenting existing or new credential options

For demonstration purposes, the option to create new credentials was selected, so that the full credential-setup process could be observed. The agent's confirmation of this selection is shown in Figure 4.3.

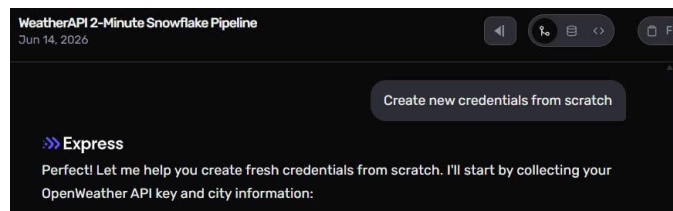


Figure 4.3: Agent confirming the creation of new credentials

4.3 Step 3 - OpenWeather API Configuration

The agent next requested the OpenWeather API key together with the name of the city for which weather data should be retrieved. Kuala Lumpur was specified as the target city, since this is the location of interest for the present exercise. Figure 4.4 shows the configuration form completed with the API key and the city name.

Figure 4.4 : OpenWeather configuration form for Kuala Lumpur

Once the credential details were submitted, the agent validated the API key against the OpenWeather service and confirmed that the connection was successful, as shown in Figure 4.5.

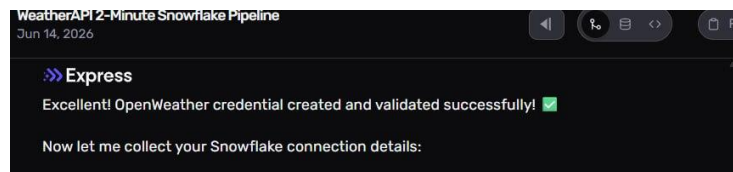


Figure 4.5: Successful validation of the OpenWeather credential

4.4 Step 4 - Snowflake Connection Setup

Snowflake is a cloud-based data warehouse service used to store and query structured data. To connect the pipeline to Snowflake, the agent requested the account identifier, login credentials, and the names of the target database, warehouse, and schema. The values supplied are listed in Table 1.

Table 1 : Snowflake Connection Parameters

Parameter	Value Supplied
Account Identifier	LXRRNWP-GZ81362
Username	SHARE
Database Name	DEMO_DB

Warehouse Name	COMPUTE_WH
Schema Name	PUBLIC

These values were entered through the configuration screens shown in Figures 4.6 and 4.7. Figure 4.6 shows the entry of the account identifier, and username.

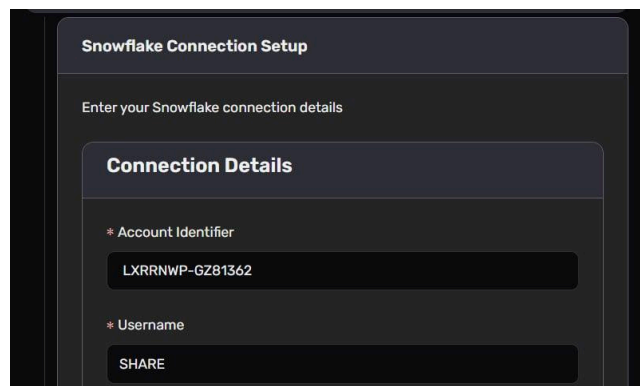


Figure 4.6: Entry of Snowflake account identifier and login details

Figure 4.7 shows the subsequent entry of the database, warehouse, and schema names that define where the data will ultimately be written.

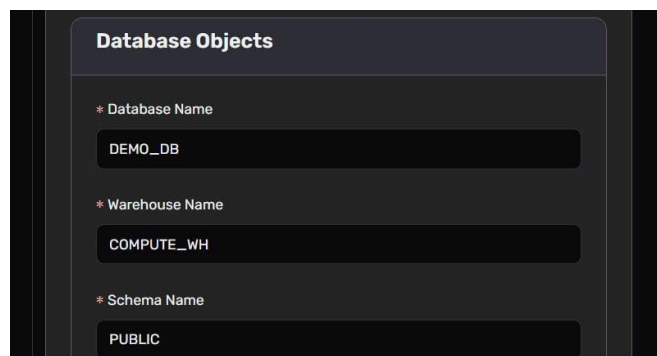


Figure 4.7: Entry of Snowflake database, warehouse, and schema

4.5 Step 5 - Source Creation and Transform Discovery

Once both the OpenWeather and Snowflake credentials had been validated, the agent automatically created the OpenWeather data source, configured to retrieve weather data for Kuala Lumpur every two minutes. The agent then performed an automatic analysis of the structure of the incoming data, referred to within the platform as Transform Discovery. Figure

4.8 shows the confirmation that both credentials were validated and that the source had been created successfully.

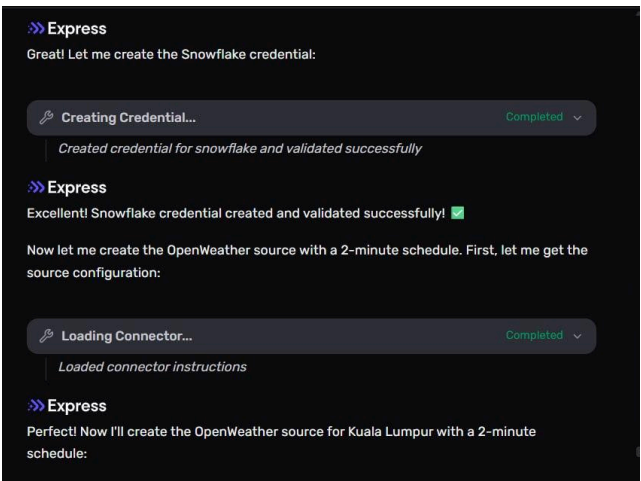


Figure 4.8: Confirmation of validated credentials and source creation

The Transform Discovery process examined the response returned by the OpenWeather API and identified its structure. The response was found to be a nested JSON object, meaning that some values are stored inside sub-objects rather than at the top level. The analysis identified the dataset as Nexset ID 428773, named “1 – OpenWeather – Kuala Lumpur Current”. The field of primary interest, `main.temp`, was located inside the main sub-object and represents the air temperature expressed in Kelvin. Other top-level fields identified in the response included `base`, `clouds`, `cod`, `coord`, `dt`, `id`, `main`, `name`, `rain`, `sys`, `timezone`, `visibility`, `weather`, and `wind`. This analysis is summarised in Figure 4.9.

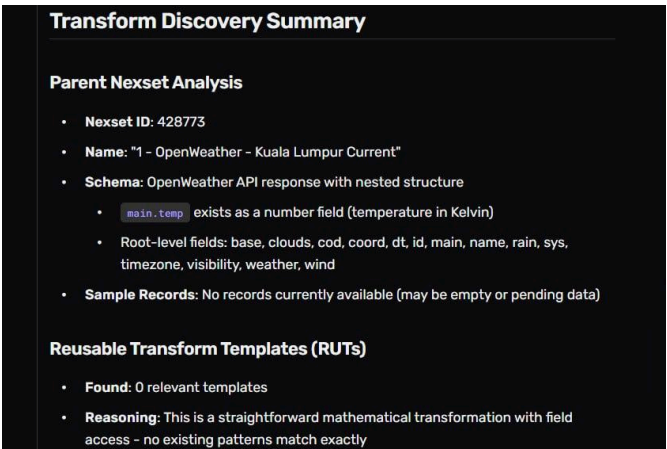
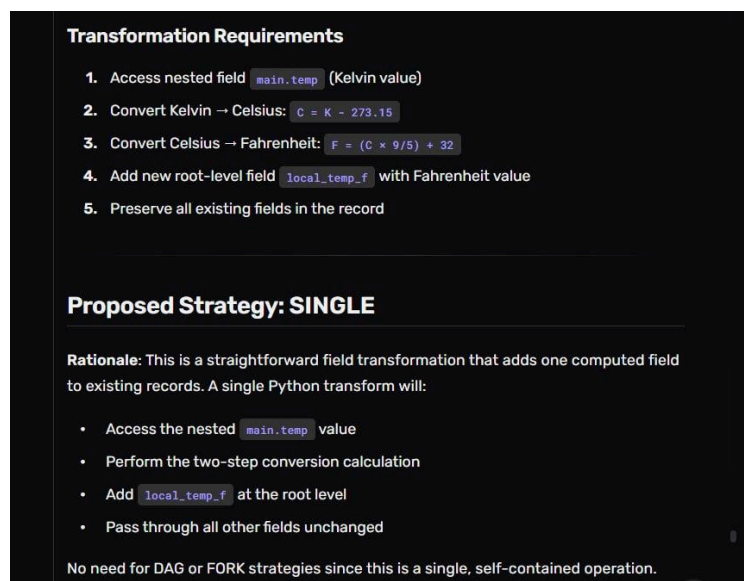


Figure 4.9: Transform Discovery Summary

4.6 Step 6 - Designing the Temperature Transformation

Based on the schema identified in the previous step, the agent proposed a transformation to convert the temperature value retrieved from the OpenWeather API, which is expressed in Kelvin, into Fahrenheit, a unit more readily understood by a general audience. The transformation was designed to perform the following operations: access the nested field `main.temp`; convert the Kelvin value to Celsius using the formula $C = K - 273.15$; convert the resulting Celsius value to Fahrenheit using the formula $F = (C \times 9/5) + 32$; add a new field, `local_temp_f`, containing the Fahrenheit result; and retain all of the original fields from the source record. These requirements are shown in Figure 4.10.



Transformation Requirements

1. Access nested field `main.temp` (Kelvin value)
2. Convert Kelvin $\rightarrow$ Celsius: $C = K - 273.15$
3. Convert Celsius $\rightarrow$ Fahrenheit: $F = (C \times 9/5) + 32$
4. Add new root-level field `local_temp_f` with Fahrenheit value
5. Preserve all existing fields in the record

Proposed Strategy: SINGLE

Rationale: This is a straightforward field transformation that adds one computed field to existing records. A single Python transform will:

- Access the nested `main.temp` value
- Perform the two-step conversion calculation
- Add `local_temp_f` at the root level
- Pass through all other fields unchanged

No need for DAG or FORK strategies since this is a single, self-contained operation.

Figure 4.10: Proposed transformation requirements for temperature conversion

To implement these requirements, the agent could choose between several transformation strategies, namely SINGLE, DAG (Directed Acyclic Graph), or FORK. A SINGLE strategy applies one transformation step in sequence without branching, whereas DAG and FORK strategies allow a record to be processed along multiple paths. Because the temperature conversion required only a straightforward sequence of arithmetic operations with no branching logic, the agent selected the SINGLE strategy. The resulting transform plan is shown in Figure 4.11.

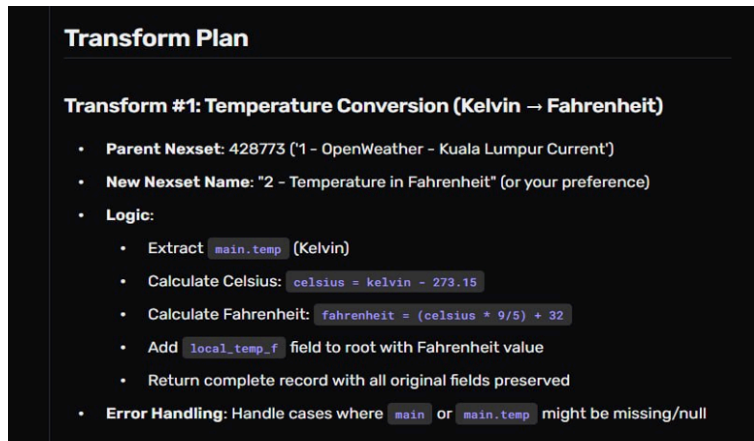


Figure 4.11: Transform plan for the Kelvin-to-Fahrenheit conversion

4.7 Step 7 - Transform Creation and Verification

The agent then automatically generated the Python code required to perform the transformation defined in Step 6. The resulting transform was assigned Nexset ID 428774 and named “2 – Temperature in Fahrenheit”. The transform added the new field `local_temp_f` to each record while preserving all of the original fields, including the complete `main` object from the source data. To handle cases where the source temperature value might be missing, the transform was designed so that if `main.temp` is null or absent, the resulting `local_temp_f` field is also set to null, rather than causing the transform to fail.

After generating the transform, the agent automatically ran a verification check and confirmed that the transform executed correctly, as shown in Figure 4.12.

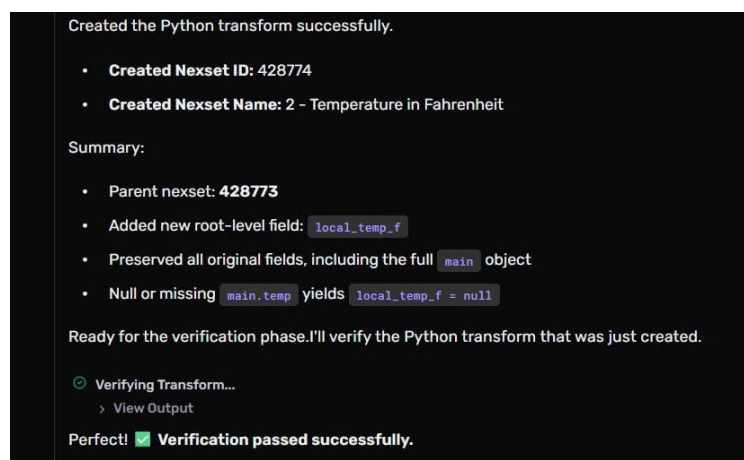


Figure 4.12: Verified Python transform for temperature conversion

4.8 Step 8 - Pipeline Summary and Activation

With the source, transformation, and destination all configured and verified, the agent compiled a summary of the complete pipeline. This summary, presented in Table 2, lists each component of the pipeline together with its configuration and status.

Table 2: Pipeline Component Summary

Component	Details	Status
Source	OpenWeather API – Kuala Lumpur, every 2 minutes Kelvin → Celsius → Fahrenheit; adds local_temp_f field Snowflake – DEMO_DB.PUBLIC.WEATHER_DATA	Active
Transform	OpenWeather API – Kuala Lumpur, every 2 minutes Kelvin → Celsius → Fahrenheit; adds local_temp_f field Snowflake – DEMO_DB.PUBLIC.WEATHER_DATA	Verified
Sink	OpenWeather API – Kuala Lumpur, every 2 minutes Kelvin → Celsius → Fahrenheit; adds local_temp_f field Snowflake – DEMO_DB.PUBLIC.WEATHER_DATA	Active

The same summary was also displayed within the agent interface, as shown in Figure 4.13, confirming that all three components of the pipeline were active and functioning as intended.

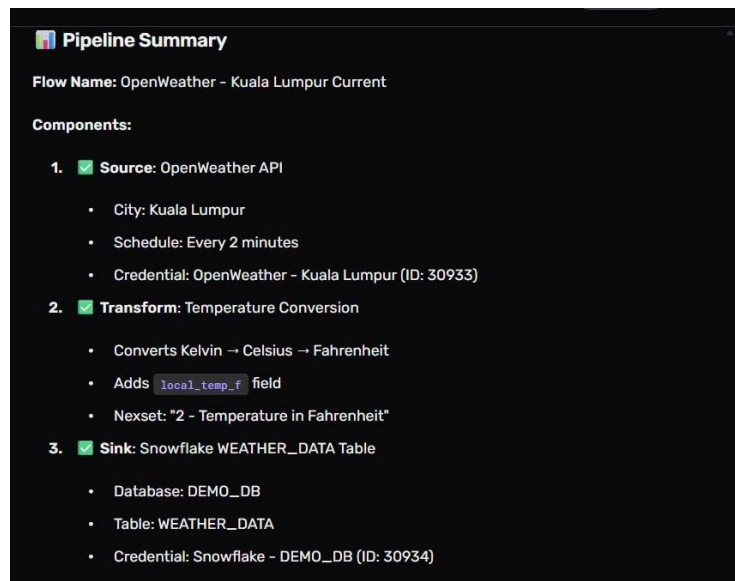


Figure 4.13: Pipeline summary displayed within the agent interface

4.9 Step 9 - Monitoring the Pipeline on Nexla

After activation, the pipeline could be observed within the Nexla Monitor dashboard. The pipeline, named “OpenWeather – Kuala Lumpur Current” and assigned Flow ID 623315, was shown to be in a RUNNING state. The dashboard displayed the pipeline as a Directed Acyclic Graph (DAG), in which each stage of the pipeline is represented as a connected node. The sequence of nodes was as follows: a scheduled trigger (CRON JOB), the OpenWeather data source (ID 123729), the initial dataset representing the raw API response (Nexset 428773), the transformed dataset containing the Fahrenheit conversion (Nexset 428774), and finally the Snowflake destination (ID 142295). Figure 4.14 shows this DAG with all nodes active.

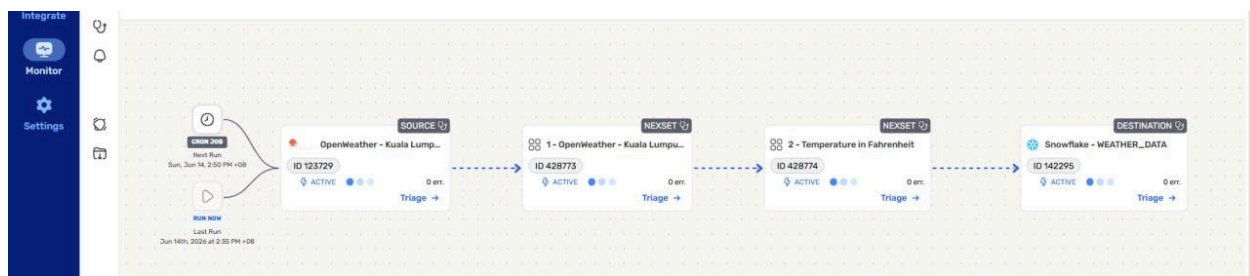


Figure 4.14: Pipeline DAG shown in the Nexla Monitor dashboard

Figure 4.15 provides an alternative view of the same pipeline within the Express agent interface, showing both the textual summary of the pipeline and a live diagram of the data flow.

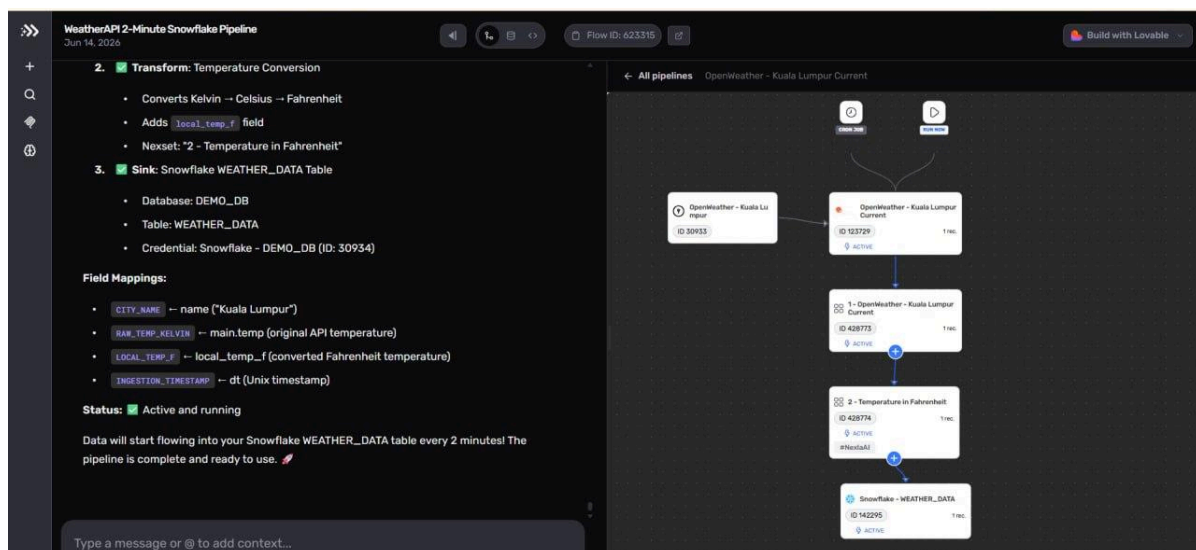


Figure 4.15: Pipeline summary and live flow diagram in the Express interface

4.10 Step 10 - Verifying Data in Snowflake

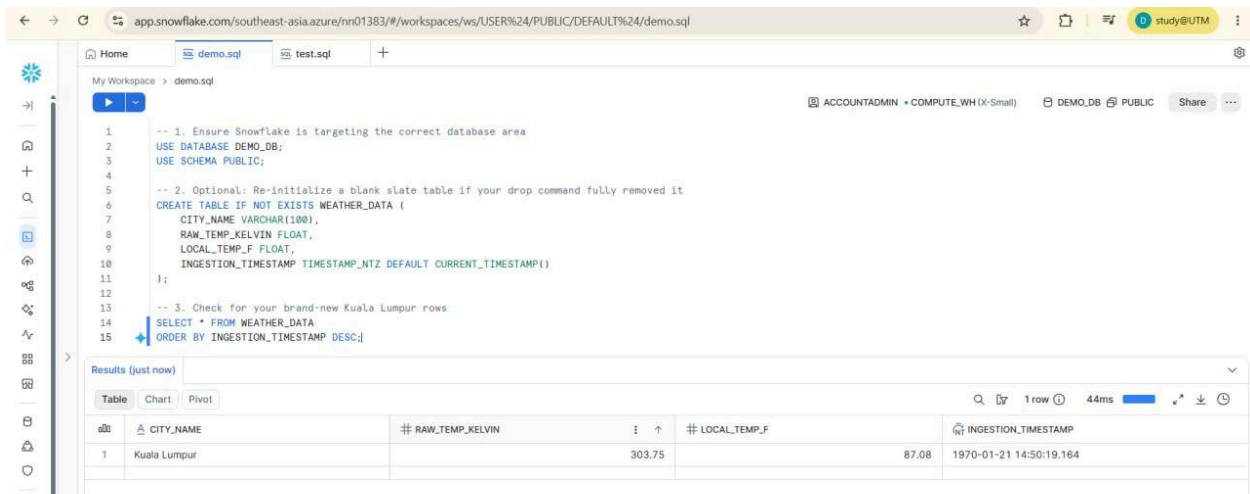
To confirm that the pipeline was correctly delivering data to its destination, a query was executed directly within Snowflake against the WEATHER_DATA table. Structured Query Language (SQL) is the standard language used to retrieve and manipulate data stored in relational databases such as Snowflake. The following statements were used to select the appropriate database and schema and to retrieve all records from the table, ordered by the time at which each record was ingested:

```
USE DATABASE DEMO_DB;
```

```
USE SCHEMA PUBLIC;
```

```
SELECT * FROM WEATHER_DATA ORDER BY INGESTION_TIMESTAMP DESC;
```

The table consists of four columns: CITY_NAME, RAW_TEMP_KELVIN, LOCAL_TEMP_F, and INGESTION_TIMESTAMP. Figure 4.16 shows the result of this query, which returned a row corresponding to Kuala Lumpur, with a raw temperature of 303.75 K converted to a value of 87.08°F.



The screenshot shows the Snowflake web interface. The SQL editor contains the following query:

```
-- 1. Ensure Snowflake is targeting the correct database area
USE DATABASE DEMO_DB;
USE SCHEMA PUBLIC;

-- 2. Optional: Re-Initialize a blank slate table if your drop command fully removed it
CREATE TABLE IF NOT EXISTS WEATHER_DATA (
  CITY_NAME VARCHAR(100),
  RAW_TEMP_KELVIN FLOAT,
  LOCAL_TEMP_F FLOAT,
  INGESTION_TIMESTAMP TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- 3. Check for your brand-new Kuala Lumpur rows
SELECT * FROM WEATHER_DATA
ORDER BY INGESTION_TIMESTAMP DESC;
```

The results pane shows a single row of data:

	CITY_NAME	RAW_TEMP_KELVIN	LOCAL_TEMP_F	INGESTION_TIMESTAMP
1	Kuala Lumpur	303.75	87.08	1970-01-21 14:50:19.164

Figure 4.16: Query result for Kuala Lumpur in the WEATHER_DATA table

Figure 4.17 shows the WEATHER_DATA table as it appears within the Snowflake Database Explorer, confirming that the table was created in the expected location, DEMO_DB.PUBLIC.

Database Explorer

DEMO_DB / PUBLIC / WEATHER_DATA

Table Details Columns Data Preview Copy History Data Quality Lineage

1 Row, Updated just now

	CITY_NAME	RAW_TEMP_KELVIN	LOCAL_TEMP_F	INGESTION_TIMESTAMP
1	Kuala Lumpur	303.75	87.08	1970-01-21T14:50

Figure 4.17: WEATHER_DATA table shown in the Snowflake Database Explorer

Finally, Figure 4.18 shows the result of the same query after the pipeline had been allowed to run for a longer period. Four rows were returned in total, including records for both Kuala Lumpur and London, confirming that the pipeline continued to operate as a recurring scheduled process rather than executing only once.

app.snowflake.com/southeast-asia.azure/n01383/#!/workspaces/ws/USER%24/PUBLIC/DEFAULT%24/test.sql

My Workspace > test.sql

```

1 -- 1. Point Snowflake to your database and schema
2 USE DATABASE DEMO_DB;
3 USE SCHEMA PUBLIC;
4
5 -- 2. Pull all the ingested weather data records
6 SELECT * FROM WEATHER_DATA
7 ORDER BY INGESTION_TIMESTAMP DESC;
8

```

Results (just now)

Table Chart Pivot

4 rows 95ms

	CITY_NAME	RAW_TEMP_KELVIN	LOCAL_TEMP_F	INGESTION_TIMESTAMP
1	null	null	57.938	2026-06-14 00:00:56.252
2	Kuala Lumpur	302.71	85.208	1970-01-21 14:50:20.382
3	London	287.56	57.938	1970-01-21 14:50:20.195
4	Kuala Lumpur	303.75	87.08	1970-01-21 14:50:19.164

Figure 4.18: Four records ingested for Kuala Lumpur and London

5.0 Results and Discussion

This section presents the outcomes of the pipeline constructed in Section 4.0, evaluates the quality of the data produced, and assesses the overall effectiveness of the AI agent's integration into the pipeline.

5.1 Pipeline Output

The pipeline constructed using the Nexla Express agent successfully demonstrated a complete, end-to-end AI-assisted data engineering workflow. Several outcomes were achieved. First, real-time data ingestion was established, with weather data for Kuala Lumpur retrieved from the OpenWeather API at a fixed two-minute interval. Second, the agent correctly identified the nested temperature field within the source data and implemented a two-step mathematical transformation, converting the value from Kelvin to Celsius and then to Fahrenheit, producing the new field `local_temp_f`. Third, the transformed data was loaded reliably into the Snowflake `WEATHER_DATA` table with the intended schema. Fourth, as evidenced in Figure 18, the pipeline continued to execute as a recurring scheduled job, producing multiple records over time rather than a single, isolated result.

5.2 Data Quality Observations

During verification, one data quality issue was identified: a single record in Figure 18 showed null values for both `CITY_NAME` and `RAW_TEMP_KELVIN`. This is likely to have resulted from an incomplete response returned by the OpenWeather API during one of the scheduled polling cycles. Although this represents a minor data quality gap, it also illustrates the value of the null-handling logic that the agent incorporated into the transformation during Step 7, whereby a missing `main.temp` value results in a null `local_temp_f` value rather than a pipeline failure. This observation highlights that, even with AI-assisted automation, downstream data quality checks remain necessary.

5.3 Assessment of AI Integration

The Express agent demonstrated effective integration capabilities across the pipeline construction process. The time required to set up the pipeline was reduced considerably, as credential validation, schema discovery, and transformation design were all automated. The agent also demonstrated an understanding of the user's intent, translating a single natural language instruction into a sequence of specific technical actions across two different systems. Error handling and verification were embedded at multiple stages, reducing the likelihood of

deploying a non-functional pipeline. Overall, the resulting pipeline followed a clear input-processing-output structure with visibility into each stage of the process.

6.0 Reflection

This tutorial demonstrates how artificial intelligence can significantly optimize data engineering workflows by building data pipelines without requiring manual coding. By utilizing the Nexla Express AI agent, a complete operational pipeline can be established solely through natural language instructions. The AI orchestrator successfully interprets intent, establishes secure connections to both the OpenWeather API and the Snowflake data warehouse, performs automatic schema discovery, and seamlessly executes mathematical transformations to convert raw temperature data from Kelvin to Fahrenheit.

However, the exercise also highlights that automated AI tools do not eliminate the necessity for strict data governance and data quality checks. During execution, an incomplete payload from the source API resulted in a record with missing values being written into the destination database. While the AI agent's proactive null-handling logic successfully prevented a total pipeline failure, this observation proves that continuous monitoring and downstream data quality validation remain mandatory to ensure analytical precision.

References

Kretz, A. (2025, November 5). *I built this complete ETL data pipeline with an AI Agent!* [Video].

YouTube. <https://www.youtube.com/watch?v=qGu7hDaJG1A>